

Inteligencia Artificial

Práctica 3

Luis Airam Saavedra Paiseo

Mayo 2026

Índice

Contenido

Índice.....	1
1. Introducción.....	2
1.1 Descripción del Problema.....	2
1.2 Diseño de la Heurística.....	2
2. Análisis de Profundidad	4
2.1 Algoritmo MiniMax.....	4
2.2 Algoritmo MiniMax con poda Alfa-Beta	6
3. Comparativa Alfa-Beta	8
4. Conclusiones.....	8
5. Anexo.....	8

Estas han sido las 2 heurísticas principales, pero se ha optado por utilizar una combinada y la primera heurística de forma independiente con un parámetro en el constructor, de forma que dependiendo de ese parámetro utilizaremos solo la heurística de los movimientos disponibles o utilizaremos ambas para la función de evaluación de los nodos terminales o nodos intermedios en caso de estar en la profundidad máxima.

```
protected int evaluar(Entorno mapa) {

    if (modoHeuristica == 1) {

        // Heurística simple: solo movilidad

        return mapa.movimientosValidos('A') -

            mapa.movimientosValidos('B');

    } else {

        // Heurística combinada: movilidad + distancia Manhattan

        int movsA = mapa.movimientosValidos('A');

        int movsB = mapa.movimientosValidos('B');

        int distA = Math.abs(mapa.getAgenteC('A') - mapa.getMetaCol('A'));

        int distB = Math.abs(mapa.getAgenteC('B') - mapa.getMetaCol('B'));

        return ((movsA - movsB) * 3) + (distB - distA);

    }

}
```

Código 1: Función de Evaluación de nodos terminales y nodos intermedios en límite de profundidad

1.2.1 Pruebas de Funcionamiento

Utilizaremos el algoritmo MiniMax con profundidad 4 para explicar el funcionamiento, este es el estado inicial:

```
#####
#A      #
#       #
#       #
#       #
#  #    B #
#####
```

En este estado,

A está en la columna 1 y su meta es la columna 15 -> $15-1=14$ (DistA)

B está en la columna 13, su meta es la columna 1 -> $13-1=12$ (DistB)

A tiene 2 movimientos válidos (S,E)

B tiene 3 movimientos válidos (N,S,O)

Por lo que si A se mueve al Este se quedaría en la columna 2 -> DistA=15-2 (13); movsA=2; movsB=3; $(2-3)*3+(12-13)=-4$.

Si A mueve al Sur quedaría en la fila 2 -> DistA=15-1 (14); movsA=3; movsB=3; $(3-3)*3+(12-14)=-2$.

El Agente elige Sur porque $-2 > -4$, en este momento moverse al sur le da más movilidad que ir al este, reflejando así una mayor utilidad esa decisión. La heurística lo refleja correctamente priorizando la movilidad sobre avanzar de forma inmediata hacia la meta. Pudiendo corroborarse con el estado 1 de esa misma ejecución:

```
#####
#a      #
#A      #
#       #
#       B #
#  #     b #
#####
```

En el que claramente se ve como lo explicado es el resultado final , verificando así de forma empírica que la heurística es correcta y el Agente decide correctamente.

2. Análisis de Profundidad

Dado que hemos utilizado una profundidad limitada, los jugadores tendrán más o menos capacidad de memoria en base a dicha profundidad, siendo así que a mayor profundidad mayor variedad de opciones a la hora de elegir que camino tomar. Esto ayudará a los jugadores a tener una mayor capacidad de ganar percibida según vayamos incrementando este valor, de forma que a cuanto más profundidad usemos, mejor será el resultado. Para demostrarlo se ha comparado el funcionamiento del jugador 'A' con el algoritmo MiniMax y con el algoritmo MiniMax con poda Alfa-Beta con profundidades "2,4,6".

2.1 Algoritmo MiniMax

Este algoritmo es la base del siguiente, se basa en la recursividad para la exploración del árbol de decisiones, de forma que el jugador sea capaz de simular los turnos del oponente así como sus turnos propios para poder decidir que es lo más óptimo para él en ese tablero simulado.

Este algoritmo ha sido implementado a partir del pseudocódigo del temario de la asignatura, quedando de manera resumida en el siguiente código:

```
public String pensar(Entorno mapa, char jugador) {

    String mejorAccion = null;

    int mejorValor = Integer.MIN_VALUE;

    for (int i = 0; i < 4; i++) {

        int f = mapa.getAgenteF(jugador);

        int c = mapa.getAgenteC(jugador);

        if (comprobarAccion(mapa, acciones[i], f+df[i], c+dc[i])) {

            mapa.moverAgente(jugador, acciones[i]);

            int valor = minValor(mapa, maxProfundidad - 1);

            mapa.deshacerMovimiento(jugador, acciones[i]);

            if (valor > mejorValor) {

                mejorValor = valor;

                mejorAccion = acciones[i];

            }

        }

    }

    return mejorAccion;

}
```

Código 2: Algoritmo MiniMax Resumido

Para poder simular todos los turnos se ha hecho uso de las funciones `maxValor` y `minValor`, también a partir del pseudocódigo del temario de la asignatura, y del pseudocódigo proporcionado en la práctica de forma auxiliar, quedando de la siguiente manera de forma resumida:

```
public int maxValor(Entorno mapa, int profundidad) {

    if (profundidad == 0 || mapa.juegoTerminadoPara('A') ||
        mapa.juegoTerminadoPara('B')) return evaluar(mapa);

    int mejorValor = Integer.MIN_VALUE;

    boolean puede = false;

    for (int i = 0; i < 4; i++) {

        int f = mapa.getAgenteF('A');

        int c = mapa.getAgenteC('A');

        if (comprobarAccion(mapa, acciones[i], f+df[i], c+dc[i])) {

            puede = true; mapa.moverAgente('A', acciones[i]);

            int valor = minValor(mapa, profundidad - 1);

            mapa.deshacerMovimiento('A', acciones[i]);

            mejorValor = Math.max(mejorValor, valor);

        }

    }

    if (!puede) return -1000;

    return mejorValor;

}
```

Código 3: Función `maxValor`

```
public int minValor(Entorno mapa, int profundidad) {

    if (profundidad == 0 || mapa.juegoTerminadoPara('A') ||
        mapa.juegoTerminadoPara('B')) return evaluar(mapa);

    int mejorValor = Integer.MAX_VALUE;

    boolean puede = false;

    for (int i = 0; i < 4; i++) {

        int f = mapa.getAgenteF('B');

        int c = mapa.getAgenteC('B');

        if (comprobarAccion(mapa, acciones[i], f+df[i], c+dc[i])) {

            puede = true; mapa.moverAgente('B', acciones[i]);

        }

    }

    if (!puede) return 1000;

    return mejorValor;

}
```

```

        int valor = maxValor(mapa, profundidad - 1);

        mapa.deshacerMovimiento('B', acciones[i]);

        mejorValor = Math.min(mejorValor, valor);

    }

}

if (!puede) return 1000;

return mejorValor;

}

```

Código 4: Función minValor

2.2 Algoritmo MiniMax con poda Alfa-Beta

Este Algoritmo es una modificación del anterior dado que tiene exactamente el mismo código y las mismas funciones minValor y maxValor, con la diferencia de que implementamos dos valores auxiliares, “Alfa y Beta”, ambos inicializados a -infinito y +infinito de forma que en cada iteración de la búsqueda de la mejor opción se irán intercambiando por los valores reales de los nodos del árbol, de forma que si en algún punto $\alpha \geq \beta$, se podará esa rama, agilizando la búsqueda y ahorrando en costes. Igual que tiene esa ventaja tiene la desventaja de que al podar podemos estar perdiendo valores de alta importancia que nos servirían de ayuda para encontrar una decisión mejor a la que hayamos tomado en ese turno.

```

public int maxValor(Entorno mapa, int profundidad, int alfa, int beta) {

    if (profundidad == 0 || mapa.juegoTerminadoPara('A') ||
        mapa.juegoTerminadoPara('B')) return evaluar(mapa);

    int mejorValor = Integer.MIN_VALUE;

    boolean puede = false;

    for (int i = 0; i < 4; i++) {

        if (comprobarAccion(mapa, acciones[i], f+df[i], c+dc[i])) {

            puede = true; mapa.moverAgente('A', acciones[i]);

            int valor = minValor(mapa, profundidad-1, alfa, beta);

            mapa.deshacerMovimiento('A', acciones[i]);

            if (valor > mejorValor) mejorValor = valor;

            if (mejorValor > alfa) alfa = mejorValor;

            if (mejorValor >= beta) break; // Poda Beta

        }

    }

    if (!puede) return -1000;

    return mejorValor;

}

```

```
}
```

Código 5: Algoritmo MiniMax con poda Alfa-Beta resumido

2.3 Pruebas de Ejecución

Estas son las pruebas detallada del jugador A con los 2 algoritmos ya mencionados a profundidades “2,4,6”:

Algoritmo	Profundidad	Ciclos hasta ganar	Nodos/turno (promedio)	Resultado
MiniMax	2	12	2-3	Victoria A
Alfa-Beta	2	17	2-3	Victoria A
MiniMax	4	12	1-36	Victoria A
Alfa-Beta	4	11	0-33	Victoria A
MiniMax	6	7	0-217	Victoria A
Alfa-Beta	6	10	0-168	Victoria A

Estado final con algoritmo **MiniMax** con profundidad 2:

```
#####  
#aa      #  
# aaaaaaaaaA #  
#          bb #  
#      Bbb bbbb #  
#  #    bbb b #  
#####
```

Estado final con algoritmo **MiniMax** con poda **Alfa-Beta** con profundidad 2:

```
#####  
#a          bbB#  
#aaaaaaaaabbbbb#  
#   aa  abbbbb#  
#   Aaaaabb bb#  
#  #      b #  
#####
```

Estado final con algoritmo **MiniMax** con profundidad 4:

```
#####  
#a      bb  #  
#aaaaaaaabb #  
#      abb  #  
#      abb  #  
#  #  AaBbbbb #  
#####
```

Estado final con algoritmo **MiniMax** con poda **Alfa-Beta** con profundidad 4:

```
#####  
#a      #  
#aaaaaaaaa bb#  
#      aBbbbb#  
#      Abb bb#  
#  #      b #  
#####
```

Estado final con algoritmo **MiniMax** con profundidad 6:

```
#####
#a          #
#aaaaaaA    bb#
#           bb#
#           bb#
#  #        bB#
#####
```

Estado final con algoritmo **MiniMax** con poda **Alfa-Beta** con profundidad **6**:

```
#####
#a          #
#aaaaaa    #
#   aa      #
#   Aa  bb  #
#  #Bbbbbbbb #
#####
```

3. Comparativa Alfa-Beta

Para comparar los resultados obtenidos por del algoritmo MiniMax sin activar la poda Alfa-Beta y activando la misma se utilizará la siguiente tabla para recoger los resultados de las 3 profundidades de cuantos nodos ha visitado en ambos algoritmos a distintas profundidades límite.

Profundidad	Nodos MiniMax (Ciclo 0)	Nodos Alfa-Beta (Ciclo 0)	Reducción %
2	2	2	0%
4	20	16	20%
6	104	77	26%

Para comparar de forma equitativa ambos algoritmos se ha utilizado el primer turno de cada partida, ya que parten del mismo tablero inicial independientemente del comportamiento del Dummy. A profundidad 2 ambos visitan el mismo número de nodos debido a que el árbol es demasiado pequeño como para que la poda tenga un efecto significativo. A profundidad 4 Alfa-Beta ya reduce un 20% el número de nodos visitados respecto a MiniMax y a profundidad 6 esto aumenta hasta un 26%, demostrándose así que cuanto mayor sea la profundidad, mayor es el beneficio de la poda Alfa-Beta al eliminar ramas que no pueden influir en la decisión final.

4. Conclusiones

En conclusión, hemos podido comprobar que el Jugador A siempre sale victorioso frente al jugador B gracias a las heurísticas utilizadas entre otras razones, así como que el ahorro al activar la poda Alfa-Beta es incremental, de forma que cuanto más quisiéramos explorar en un solo turno, mayor sería el beneficio de esta, así como mayor sería el riesgo de perder información importante de ramas podadas, pero es un riesgo a asumir puesto que de no utilizarse la poda, el tiempo de ejecución así como los nodos visitados podrían crecer de manera exponencial llegando a consumir el tiempo límite de ejecución en algunos casos.

5. Anexo

Una mejora significativa sería implementar la heurística de **Territorios Voronoi**, que calcula cuántas casillas del tablero son más accesibles para A que para B. Con esto el agente no solo evalúa sus movimientos inmediatos, sino que planifica el control del espacio, resultando en una estrategia de acorralamiento más efectiva.

Otra mejora sería ajustar el orden de exploración de movimientos en maxValor y minValor. Actualmente se exploran siempre en orden fijo N,S,E,O, lo que limita la efectividad de la poda Alfa-Beta. Si se ordenaran los movimientos por su valor heurístico antes de explorarlos, la poda sería mucho más agresiva y la reducción de nodos visitados podría superar el 50%.